Proof

$4n^2 + 7n + 6 = O(n^2)$

$n \geq 1$
$n^2 \geq n \Rightarrow 7n^2 \geq 7n$
$n^2 \geq 1 \Rightarrow 6n^2 \geq 6$
$4n^2 + 7n^2 + 6n^2 \geq 4n^2 + 7n + 6$
$17n^2 \geq 4n^2 + 7n + 6$

$\therefore 4n^2 + 7n + 6 = O(n^2)$, $n_0 = 1$ and $c = 17$

Proof w/ nontrivial n_0

$4n^2 + 7n + 6 = O(n^2)$

$n \geq 7$
$n^2 \geq 7n$
$n^2 \geq 49 \Rightarrow n^2 \geq 6$
$4n^2 + n^2 + n^2 \geq 4n^2 + 7n + 6$
$6n^2 \geq 4n^2 + 7n + 6$
$\therefore 4n^2 + 7n + 6 = O(n^2)$, $n_0 = 7$ and $c = 6$

Proof Big-Omega

$\frac{3}{4}n^2 - 6n = \Omega(n^2)$

Set $\frac{3}{4}n^2 - 6n \geq cn^2$, for all $n \geq n_0$
$6n \leq \frac{3}{8}n^2 \Rightarrow 6 \leq \frac{3}{8}n \Rightarrow 16 \leq n \Rightarrow n_0 = 16$
$\frac{3}{4}n^2 - 6n \geq \frac{3}{4}n^2 - \frac{3}{8}n^2 \Rightarrow \frac{3}{4}n^2 - 6n \geq \frac{3}{8}n^2$
$\therefore \frac{3}{4}n^2 - 6n = \Omega(n^2)$, $n_0 = 16$ and $c = \frac{3}{8}$

Big-Theta Proof

$f(n) = 5n^2 + 8n + 3 = \Theta(n^2)$

$n \geq 1 \Rightarrow n^2 \geq 1 \Rightarrow 3n^2 \geq 3$
$n^2 \geq n \Rightarrow 8n^2 \geq 8n$
$5n^2 + 8n^2 + 3n^2 \geq 5n^2 + 8n + 3$
$16n^2 \geq 5n^2 + 8n + 3$
$f(n) = O(n^2)$, $n_{0,1} = 1$, and $c_2 = 16$

$5n^2 + 8n + 3 \geq 5n^2$ for $n \geq 1$
$f(n) = \Omega(n^2)$, $n_{0,2} = 1$, and $c_1 = 5$

$n_0 = \max(n_{0,1}, n_{0,2}) = 1$
$c_1 = 5$, $c_2 = 16$
$f(n) = \Theta(n^2)$

Answer the following questions

$$fib(n) = fib(n-1) + fib(n-2)$$

$$fib(0) = fib(1) = 1$$

1.

$$fib(5) = fib(4) + fib(3)$$

$$= fib(3) + fib(2) + fib(2) + fib(1)$$

$$= fib(2) + fib(1) + fib(2) + fib(2) + fib(1)$$

$$= 5$$

2.

$$fib(6) = fib(5) + fib(4)$$

$$= 5 + fib(3) + fib(2)$$

$$= 5 + fib(2) + fib(1) + fib(2)$$

$$= 5 + 3 = 8$$

3.

$$fib(7) = fib(6) + fib(5) = 8 + 5 = 13$$

4.

$$fib(8) = fib(7) + fib(6) = 13 + 8 = 21$$

5.

$$fib(9) = fib(8) + fib(7) = 21 + 13 = 34$$

---

1. Explain what the code above is doing.
   A: It is a recursive function. It calls for the fibonacci value one below and two below the integer that was put in. And it then calculates that fibonacci value by calling the 2 values below it, and on and on until it hits 1 or 0, and then it sums all the results.
2. What happens if we remove the "if ... return ..." and only keep the last line?
   A: It won't stop (and then just error out). Because the recursive calls never end, it will be calling fibonacci(-20) and that doesn't make sense.
3. What is fibonacci(20)? how much time did it take to calculate that?
   A: fibonacci(20) = 6765, calculating this took 1.2567e-03 seconds.
4. What is fibonacci(30)? how much time did it take to calculate that?
   A: fibonacci(30) = 832040, calculating this took 8.0916e-02 seconds.
5. How much time did it take you to calculate fibonacci(40)? (this might take a while...)
   A: fibonacci(40) = 102334155, calculating this took 7.1348e+00 seconds.

How many function calls?

1. How many function calls does fibonacci(1) take?
   A: 1 call.
2. How many function calls does fibonacci(5) take?
   A: 15 calls.
3. How many function calls does fibonacci(10) take?
   A: 177 calls.
4. Why is it so slow? Where does the complexity come from?
   A: The two recursive branches repeatedly calculate the same Fibonacci values. The number of calls grows exponentially because those branches do not share their results.
5. Is this O(n)? is this O(2^n)? Why?
   A: It is not $O(n)$. It is $O(2^n)$ because each call creates at most two recursive calls, and the recursion has at most n levels. The total number of calls is bounded by $1 + 2 + \cdots + 2^n = O(2^n)$.
6. Is this Ω(n)? Why?
   A: Yes. One chain of recursive calls goes from n down to 1, so there are at least n calls. An exponential running time can also have a linear lower bound.

Memoization Optimization

1. How is this one different from the previous one?
   A: It stores calculated Fibonacci values in a cache. If a later call needs the same value, it returns the cached result instead of calculating it again.
2. How much time does it take to calculate fibonacci(30)?
   A: fibonacci(30) = 832040. One measured run on this computer took 4.3341e-06 seconds, starting with an empty cache.
3. Why is it often faster?
   A: Each distinct non-base Fibonacci value is calculated once. Later calls reuse the saved result, avoiding the repeated recursive work of the basic implementation.
4. Also modify this file to count: how many times the function had to be called for fibonacci(30)?
   A: 59 calls, including calls that return a base-case value or a cached result. The counter is in the memoized implementation.
5. Is this O(n)? is this O(2^n)? Why?
   A: Yes to both. With an empty cache, it performs $O(n)$ calls and constant work per call under the usual arithmetic cost model. Since $n \leq 2^n$ for positive n, $O(2^n)$ is also a valid but looser upper bound.
6. Is this Ω(n)? is this Ω(2^n)? Why?
   A: It is $\Omega(n)$ because it must calculate all the Fibonacci values from 2 through n when the cache starts empty. It is not $\Omega(2^n)$ because its running time is $\Theta(n)$, which grows more slowly than 2^n.

Extension: Staircase Problem

The implementation keeps the previous three values and adds them to calculate the next one. It uses $O(n)$ time and $O(1)$ extra space under the usual arithmetic cost model.

fibonacci_threeway(50) = 4045078385041. One measured run took 1.5418e-06 seconds, below the one-second requirement.